

Bilgisayar Mühendisliği Bölümü
BIMU2057 Veri Yapıları

Dr. Öğr. Üyesi Nihal ALTUNTAŞ

nihal.altuntas@iuc.edu.tr

Bugün Neleri Hatırlayacağız?

- Değişkenler ve temel veri tipleri
- List, tuple, set ve dictionary
- Koşullar ve döngüler
- Fonksiyonlar
- Mutable / immutable nesneler
- Atama, referans davranışı ve kopyalama
- Sınıflar ve nesneler
- Veri yapılarına hazırlık

Python'da Veri Tipleri

- Python dinamik tipli bir dildir; değişken tanımlarken tip belirtmeye gerek yoktur.
  - Bir değişkenin tipi, çalışma zamanında type() ile öğrenilebilir.

- Temel tipler: int, float, complex, bool, str
- Koleksiyon tipleri: list, tuple, dict, set
- Boş değer: None

Değişkenler ve Dinamik Tip Sistemi

- Python'da değişken tanımlarken veri tipi ayrıca belirtilmez.

```
x = 10
print(x)
x = 3.14
print(x)
x = "Data Structures"
print(x)
```

```
a = 5
b = a
b = 20
print(a)
print(b)
```

Çıktı ne olur?

Temel Veri Tipleri

```
student_no = 123
average = 78.5
name = "Ada"
passed = True
nothing = None

print(type(student_no))
print(type(average))
print(type(name))
print(type(passed))
print(type(nothing))
```

Çıktı ne olur?

Listeler

- Listeler ilişkili verileri bir arada tutan yapılardır.
  - Python'un Array'i
- Dinamiktirler; boyutları program boyunca değişebilir.
- Farklı tiplerde elemanlar aynı listede tutulabilir.
- Temel işlemler
  - append()
  - insert()
  - remove()
  - pop()
  - slicing [a:b]

Listeler

```
grades = [80, 70, 90]

print(grades[0])
grades.append(100)
grades[1] = 75

for grade in grades:
    print(grade)

print(len(grades))
```

Çıktı ne olur?

Listeler

```
a = [1, 2, 3]
b = a
b.append(4)

print("a:", a)
print("b:", b)
```

Çıktı ne olur?

Listeler

- Indexing ve Slicing

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])
print(numbers[-1])

print(numbers[1:4])
print(numbers[:3])
print(numbers[::2])
```

Çıktı ne olur?

List, Tuple, Set, Dictionary

- List
  - Değiştirilebilir: [10, 20, 30]
- Tuple
  - Değiştirilemez: (10, 20, 30)
- Set
  - Benzersiz elemanlar: {10, 20, 30}
- Dict
  - Anahtar-değer çiftleri: {'name': 'Ada', 'grade': 90}
- Her yapı farklı bir probleme daha uygun olabilir.

Dictionary Örneği

```
student = {
    "no": 123,
    "name": "Ada",
    "grades": [80, 90, 75]
}

print(student["name"])
student["average"] = sum(student["grades"]) / len(student["grades"])

for key, value in student.items():
    print(key, value)
```

Koşullar

```
grade = 72

if grade >= 90:
    letter = "AA"
elif grade >= 70:
    letter = "BB"
elif grade >= 50:
    letter = "CC"
else:
    letter = "FF"

print(letter)
```

Döngüler

```
numbers = [4, 7, 2, 9, 5]

total = 0
for x in numbers:
    total += x

print(total)

i = 0
while i < len(numbers):
    print(i, numbers[i])
    i += 1
```

Fonksiyonlar

- Hazır max() fonksiyonunu kullanmadan maksimum değeri bulalım.

```
def find_max(values):
    maximum = values[0]

    for value in values[1:]:
        if value > maximum:
            maximum = value

    return maximum

numbers = [4, 7, 2, 9, 5]
print(find_max(numbers))
```

Önemli: Mutable ve Immutable

- İki örneğin davranışı neden farklı?

```
x = 5
y = x
y += 1
print(x, y)

a = [1, 2, 3]
b = a
b.append(4)
print(a)
print(b)
```

Atama vs Kopyalama

- b = a yeni bir liste oluşturmaz; iki isim aynı liste nesnesini gösterir.
- Basit listelerde copy() veya slicing ile ayrı bir liste oluşturabiliriz.

```
a = [10, 20, 30]
b = a
c = a.copy()
d = a[:]

print(a is b)
print(a is c)
print(a is d)

print(a == b)
print(a == c)
print(a == d)
```

```
b[0] = 99
c[1] = 88
d[2] = 77

print(a)
print(b)
print(c)
print(d)
```

Çıktı ne olur?

Fonksiyona Mutable Nesne Gönderirsek...

```
def add_grade(grades, grade):
    grades.append(grade)

student_grades = [70, 80]
add_grade(student_grades, 90)

print(student_grades)
```

Çıktı ne olur?

Sınıflar ve Nesneler

- ***Veri yapılarında düğüm (node) gibi yapıları sınıflarla temsil edeceğiz.***

```
class Student:
    def __init__(self, no, name):
        self.no = no
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)

student = Student(123, "Ada")
student.add_grade(85)

print(student.name)
print(student.grades)
```

Veri Yapılarına İlk Bakış: Node

- Buradaki next başka bir nesneye referans tutar.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

first = Node(10)
second = Node(20)

first.next = second

print(first.data)
print(first.next.data)
```

Çıktı ne olur?